

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет по курсовой работе
по дисциплине «Курсовое проектирование по управлению ресурсами
суперэвм»

«Решение задачи возведения матрицы в k -тую степень»

Студент,
группы 5130201/30101

_____ Мартыненко А. П.

Преподаватель

_____ Курочкин М. А.

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Постановка задачи	4
2 Аппаратно-программная платформа Nvidia CUDA	5
2.1 Архитектура Nvidia CUDA	5
2.2 Вычислительные возможности Nvidia CUDA	6
2.3 Поточковая модель	7
2.4 Устройство памяти	8
2.5 Модели памяти	9
2.6 Модель вычислений на GPU	11
2.7 Компиляция программы	11
2.8 Планировщик задач	12
3 Суперкомпьютерный центр «Политехнический»	13
3.1 Состав	13
3.2 Характеристики	13
3.3 Технология подключения	14
4 Постановка решаемой практической задачи	16
5 Алгоритм решения задачи	17
5.1 Метод распараллеливания алгоритма	17
5.1.1 Глобальная память	17
5.1.2 Разделяемая память	18
6 Описание эксперимента	20
7 Анализ результатов	21
Заключение	29
Список использованной литературы	30

Введение

Сейчас многие научные и инженерные задачи требуют больших мощностей машин, на которых они выполняются, так как в ходе их выполнения необходимо обрабатывать огромное число математических операций за приемлемое время. Одним из вариантов решения сложных вычислительных задач является использование параллельного программирования.

Суперкомпьютеры представляют собой многопроцессорные вычислительные комплексы, способные выполнять триллионы операций в секунду. Их использование позволяет решать задачи, недоступные для традиционных последовательных вычислений. Это особенно актуально для таких областей, как численное моделирование, обработка изображений, машинное обучение. В связи с этим в последние годы стало очень распространено вычисление с помощью графических ускорителей — устройств с массивно-параллельной архитектурой. Производить вычисления общего назначения можно на видеокартах архитектуры Nvidia CUDA.

Одним из примеров массивных вычислительных систем является суперкомпьютерный центр «Политехнический». Часть узлов этого суперкомпьютера оборудована графическими ускорителями Nvidia Tesla K40X.

1 Постановка задачи

Цель курсовой работы: исследование технологий параллельного программирования на основе архитектуры NVIDIA CUDA для решения задачи возведения матрицы в k -тую степень с использованием вычислительных ресурсов СКЦ «Политехнический».

В рамках поставленной цели необходимо выполнить следующие задачи:

1. Изучить аппаратно-программную платформу NVIDIA CUDA: её архитектуру, вычислительные возможности, устройство памяти, модель вычислений.
2. Изучить состав и характеристики вычислительных средств СКЦ «Политехнический», изучить и применить технологию подключения к СКЦ.
3. Реализовать с использованием технологии CUDA C алгоритм возведения матрицы в k -тую степень.
4. Провести серию вычислительных экспериментов на узле «Торнадо» СКЦ «Политехнический» с изменением следующих параметров:
 - количество потоков в блоке,
 - количество блоков,
 - размер исходной матрицы,
 - используемые модели матрицы.
5. Провести анализ полученных результатов и сравнить время выполнения реализации при различных параметрах.

2 Аппаратно-программная платформа Nvidia CUDA

NVIDIA CUDA (Compute Unified Device Architecture) — это программная и аппаратная платформой от NVIDIA, которая позволяет использовать графические процессоры (GPU) для параллельных вычислений и ускорения обработки данных.

2.1 Архитектура Nvidia CUDA

Архитектура GPU включает:

- Процессор GPU, который представляет собой массив потоковых процессоров (Streaming Processor Array);
- Потоковый процессор, который состоит из кластеров текстурных процессоров (Texture Processor Clusters);
- Текстурные процессоры, который состоят из набора мультипроцессоров (Streaming Multiprocessor);
- Мультипроцессоры, которые содержат несколько потоковых процессоров (Streaming Processors) или ядер (CUDA cores).

Планирование выполнения команд происходит при помощи GigaThread Engine. Он распределяет блоки потоков по мультипроцессорам. Общий вид GPU кардинально не меняется при переходе от одной микроархитектуры к другой, изменяется размер и скорость L2 кэша.

В архитектуре Nvidia CUDA применяется SIMT (Single Instruction Multiple Thread) модель исполнения. Это модель является комбинацией из MIMD (Multiple Instruction Multiple Data) и SIMD (Single Instruction Multiple Data). Задача разбивается на множество блоков. Планировщик распределяет блоки по SM (вычислительным блокам), при этом каждый блок выполняется только на одном SM, но на одном SM может выполняться несколько блоков.

Каждый блок состоит из потоков (threads), при исполнении потоки разбиваются на варпы — группы по 32 потока. Все потоки варпа выполняют в одно время одну и ту же инструкцию.

2.2 Вычислительные возможности Nvidia CUDA

Видеокарты Nvidia CUDA разных микроархитектур обладают разным количеством ядер разного назначения. Ниже приведены краткие сведения о вычислительных возможностях различных поколений Nvidia CUDA.

Микроархитектура Fermi

- В каждом потоковом процессоре (SM):
 - 32 CUDA Core (ядра CUDA для выполнения операций с целыми числами и с числами с плавающей точкой);
 - 16 LD/ST блоков (ядер загрузки/выгрузки данных);
 - 4 SFU - блока специального назначения (для вычисления сложных арифметических функций).
- Планировщики: 2 Warp Scheduler с dual-issue (параллельное выполнение независимых инструкций).
- Память: регистровый файл, 128KB; L1-cache, 16KB/48KB; разделяемая память, 48KB/16KB.
- Особенности: базовая структура SM, поддержка dual-issue для повышения производительности.

Микроархитектура Kepler

- SMX: 192 CUDA Core (12 групп по 16 ядер), 64 блока двойной точности (FP64), 32 LD/ST, 32 SFU.
- Планировщики: 4 Warp Scheduler с двумя блоками отправки команд каждый.
- Память: регистровый файл, 256KB; L1-cache, 16KB/48KB/32KB; разделяемая память, 48KB/16KB/32KB; константная память (Read-Only Data Cache), 48KB.

Микроархитектура Maxwell

- SMM: 4 блока по 32 CUDA Core, 8 LD/ST блоков, 8 SFU блока.
- Память: 64KB регистровый файл на блок, 90KB Shared Memory (отдельный блок).

- Особенности: Модульная структура, улучшение производительности на ватт в 2 раза, на площадь кристалла — в 1.4 раза.

Микроархитектура Pascal

- SM: 2 блока с удвоенным регистровым файлом и увеличенным числом блоков двойной точности.
- Память: 64KB Shared Memory, Texture/L1 кэш.
- Особенности: Переход на 16нм FinFET, улучшение производительности на ватт.

Микроархитектура Turing

- SM: 4 блока, каждый содержит 32 FP32 ядра, 32 INT ядра, 8 FP64 ядер, 2 Tensor Core, 8 LD/ST, 1 SFU.
- Tensor Core: Обработка матриц 4x4 (FP16/FP32 вход, FP32/FP64 выход), прирост до 9.3x в задачах глубокого обучения.
- Память: 128KB L1/Shared Memory (до 96KB для Shared Memory).
- Особенности: Independent Thread Scheduling (у каждого потока свой счетчик команд), 12нм FinFET, Tensor Core для ИИ.

2.3 Потокковая модель

Вычислительная архитектура CUDA основана на концепции мульти-процессора и SIMT (Single Instruction Multiple Threads) — одна команда на множество данных.

При выполнении многопоточной программы на видеокарте CUDA, все потоки разделяются на блоки, а внутри блоков на варпы, где все потоки выполняют одну и ту же инструкцию. Группа блоков выполняется на потоковом процессоре, распределением задач занимается планировщик.

Программа, выполняющаяся на нескольких блоках одновременно называется ядром (kernel).

Особенностью архитектуры CUDA является блочно-сеточная организация, необычная для многопоточных приложений. Данная организация представлена на рисунке 1. При этом драйвер CUDA самостоятельно распределяет ресурсы устройства между потоками.

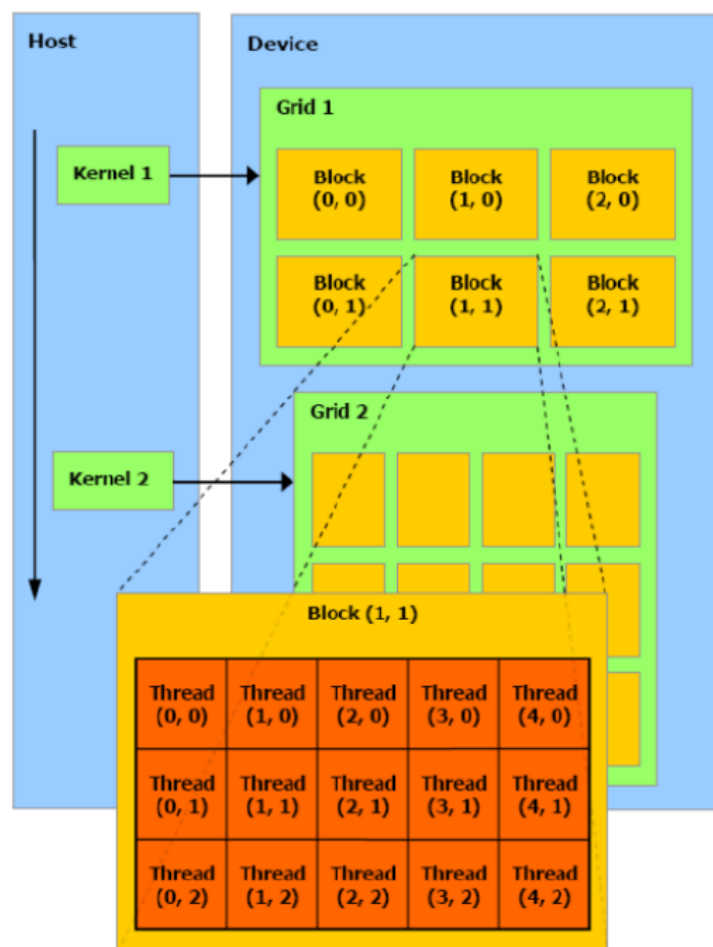


Рис. 1. Организация потоков

2.4 Устройство памяти

Видеокарта имеет собственную оперативную глобальную память, отдельную от оперативной памяти CPU.

Когда на видеокарте выполняется код, доступ возможен только к её собственной памяти. Поэтому для выполнения вычислений и получения результата необходимо перенести данные из оперативной памяти ЦП в память видеокарты и обратно. Помимо глобальной оперативной памяти на каждом мультипроцессоре есть свой кэш, своя текстурная, константная, разделяемая память и тд. Схема организации памяти CUDA-программы представлена на рисунке 2.

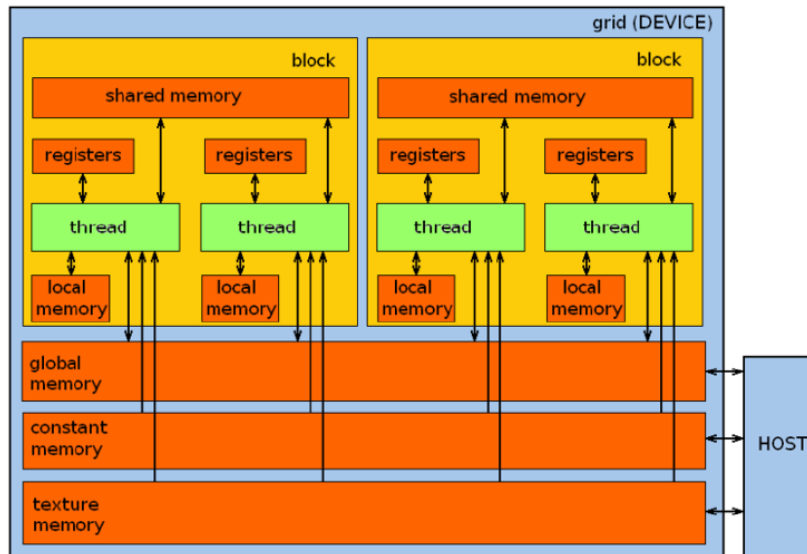


Рис. 2. Схема организации памяти CUDA-программы

2.5 Модели памяти

В CUDA выделяют шесть видов памяти:

- **Регистры**

Это память, в которой по возможности компилятор старается размещать все локальные переменные функций. На один мультипроцессор доступно 8192 32-разрядных регистров. Они распределяются между нитями в этом потоке. Обращение к этой памяти самое быстрое.

- **Локальная память.**

Это небольшой объём памяти, к которому имеет доступ только один потоковый процессор. В этот тип памяти помещаются локальные данные, которые имеют слишком большой размер. Физически локальная память является аналогом глобальной памяти, и работает с той же скоростью.

- **Глобальная память.**

Глобальная память имеет большой объём (размер от 256Мбайт до 4Гбайт). Она поддерживает произвольный доступ для всех мультипроцессоров, а также запись и чтение с хоста. Однако, эта память очень медленная и не кэшируется, поэтому рекомендуется сократить количество обращений к этой памяти. Глобальная память находится на хосте, в то время как разделяемая память расположена на плате. Это приводит к тому, что накладные расходы на переда-

чу данных по каналам связи для глобальной памяти выше, чем для разделяемой памяти, хотя сами по себе они идентичны.

- **Разделяемая память.**

Разделяемая память — это некешируемая, но быстрая память. Ее рекомендуется использовать в качестве управляемого кэша. На один мультипроцессор доступно всего 16КВ разделяемой памяти. Она обеспечивает взаимодействие потоков, управляется разработчиком напрямую и имеет низкие задержки. Отличительной чертой разделяемой памяти является то, что она адресуется одинаково для всех задач внутри блока, то есть ее можно использовать для обмена данными между потоками только одного блока.

- **Константная память.**

Константная память — это тип памяти, который хранит неизменяемые данные, доступные на уровне сетки. Ее размер 64КВ, физически она не отделена от глобальной памяти. Однако за счет того, что она использует системы кэшей и механизм широкого вещания (broadcast), данный тип может обеспечить прирост производительности за счет сокращения трафика между процессором и памятью. Однако из-за её небольшого объема имеет смысл хранить лишь небольшое количество часто используемых данных.

- **Текстурная память.**

Текстурная память — это тип памяти, который похож на константную память, поскольку через текстурный блок разрешены запросы только для чтения. Физически текстурная память не отделена от глобальной. Как и константная память, позволяет увеличить производительность за счет системы кэшей. Отличительной особенностью является оптимизация текстурного кэша для двумерной пространственной локальности.

При вычислении на модели в виде матрицы, где соседние элементы взаимодействуют друг с другом, часто возникает необходимость обращаться к элементам окрестности элемента матрицы. С точки зрения адресной арифметики элементы окрестности какого-либо элемента матрицы не расположены в памяти рядом друг с другом. Данный вид памяти позволяет кешировать данные по свойству их пространственной, а не адресной локальности, то есть те данные, которые расположены рядом в двумерном пространстве.

2.6 Модель вычислений на GPU

Программа, запускаемая на GPU с Nvidia CUDA, называется ядром (kernel). Ядро запускается одновременно на сетке из блоков. Каждый из блоков состоит из нескольких нитей. Количество блоков и потоков задается при вызове ядра. В каждой нити выполняется один и тот же код. Добиться разного поведения в нитях возможно при помощи информации о номере блока и нити, которая доступна для каждой нити.

С точки зрения GPU потоков не существует, и все команды, поступающие на GPU, выполняются в порядке общей очереди для всех потоков.

Планировщик может сгладить последовательность выполнения команд, запустив одновременно копирование с хоста, выполнение ядра и копирование на хост. Если ядро использует менее 50% мощности GPU, планировщик может запустить следующее ядро из другого потока, если оно готово к выполнению.

Для оптимизации времени выполнения задач в разных потоках необходимо учитывать, что на GPU команды выполняются в том порядке, в котором они были вызваны в коде хоста. Это означает, что не всегда лучше заполнять задачами один поток, затем второй и так далее. Более оптимальным подходом будет равномерный запуск задач по всем потокам.

Для этого можно сначала заполнить один поток командами, а затем другой. Тогда команды второго потока будут ожидать окончания выполнения команд первого потока. Другой вариант, когда команды будут распределяться по потокам поочередно: в первый поток отправляется первая команда, во второй отправляется также первая, затем в первый вторая, и т.д. В результате такого равномерного распределения можно добиться улучшения производительности за счет умения планировщика одновременно запускать операции копирования и ядра.

2.7 Компиляция программы

Программа для видеокарт Nvidia CUDA пишется на основе других языков, в частности используется расширение языка C. Оно называется CUDA C.

Для сборки программы используется компилятор nvcc, который входит в пакет инструментов разработчика. Этот пакет, а также библиотеку CUDA можно скачать с сайта Nvidia.

2.8 Планировщик задач

Поток (stream) в CUDA — логическая последовательность зависимых асинхронных операций, независимая от операций в других потоках. Потоки позволяют запускать CUDA команды на GPU в порядке, определенном в контексте одного потока.

С точки зрения GPU потоки не существуют как отдельные сущности, и все команды, пришедшие на GPU, будут исполняться в порядке общей для всех потоков очереди, знание этой особенности может помочь при оптимизации.

Последовательность исполнения может сгладить планировщик, который может запустить одновременно копирование с хоста, на хост и исполнение ядра. При вызове ядра можно указать поток, в который будет добавлено это ядро. По умолчанию все ядра добавляются в поток 0, который является синхронным с хостом. Если ядро использует менее 50% мощности GPU, то планировщик может запустить следующее ядро из другого потока, если оно готово к выполнению.

Планировщик задач в CUDA — это механизм, который управляет выполнением потоков на GPU. Он отвечает за распределение ресурсов, таких как вычислительные ядра и память, между потоками и блоками. На рисунке 3 показан планировщик задач для архитектуры Kepler.

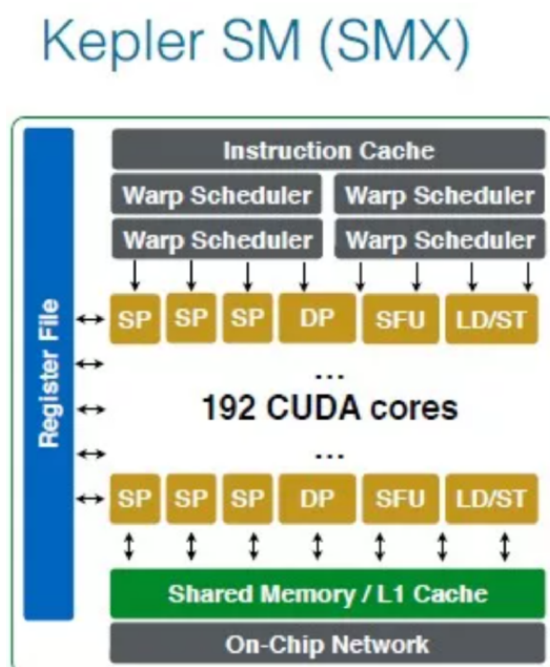


Рис. 3. Планировщик задач в архитектуре Kepler

3 Суперкомпьютерный центр «Политехнический»

3.1 Состав

Пользователям СКЦ «Политехнический» в настоящее время доступны следующие вычислительные ресурсы:

- 612 узлов кластера «Политехник - РСК Торнадо»;
- 56 узлов кластера «Политехник - РСК Торнадо» с ускорителями NVIDIA K-40.
- 288 узлов вычислителя с ультравысокой многопоточностью «Политехник - РСК ПетаСтрим»;

3.2 Характеристики

- один узел кластера «Политехник - РСК Торнадо» содержит:
 - 2 x Intel Xeon CPU E5-2697 v3 @ 2.60GHz
 - 64G RAM
- один узел кластера «Политехник - РСК Торнадо» с ускорителями NVIDIA K-40 содержит:
 - 2 x Intel Xeon CPU E5-2697 v3 @ 2.60GHz
 - 64G RAM
 - 2 x Nvidia Tesla K40x 12G GDDR
- один узел кластера «Политехник - РСК ПетаСтрим» содержит:
 - 1 x Intel Xeon Phi 5120D @ 1.10GHz
 - 8G RAM

Все доступные узлы используют сеть 56Gbps FDR Infiniband в качестве интерконнекта. Также, на всех узлах доступна параллельная файловая система Lustre объёмом около 1 ПБ.

По умолчанию пользователю предоставляется доступ к узлам tornado. Доступ к остальным типам узлов предоставляется по запросу.

3.3 Технология подключения

Для подключения зарегистрированного пользователя к СКЦ необходимо использовать любой терминальный клиент, поддерживающий протокол SSH. С помощью него получается доступ к удаленному терминалу для работы с ресурсами СКЦ.

В рамках работы была использована следующая технология подключения:

1. От администрации СКЦ были получены публичный и приватный ключи в виде файлов, а также логин пользователя `tm4u29` для доступа к системе.
2. Установка программного обеспечения:

- Для удобства переноса файлов между персональным компьютером и узлом СКЦ был установлен клиент WinSCP.
- Для организации подключения к терминалу СКЦ необходимо установить инструмент OpenSSH.

На ноутбуке с операционной системой Windows 11 OpenSSH предустановлен, однако нужно запустить ручную службу `ssh-agent`. Для этого нужно запустить `cmd` от имени администратора и выполнить команды

```
1 Set-Service ssh-agent -StartupType Automatic
2 Start-Service ssh-agent
```

Далее нужно добавить ключ

```
1 ssh -add path /to/ id_rsa # Список добавленных ключей
2 ssh-add -l
```

3. Подключение к узлу СКЦ через SSH: выполнить команду

```
1 ssh tm4u29@login1.hpc.spbstu.ru -i ~/.ssh.martynenko3.ap
```

где

- `tm4u29` - логин,
- `login1.hpc.spbstu.ru` - адрес узла СКЦ,
- `-i ~/.ssh.martynenko3.ap` - путь до файла приватного ключа.

4. Настройка WinSCP для передачи файлов: настроить следующие параметры

-

В результате было осуществлено подключение к СКЦ по протоколу SSH. На рисунке 5 представлена стартовая страница при подключении к СКЦ.

[illegible]

15

4 Постановка решаемой практической задачи

Дано:

- Массив M - матрица $n \times n$

Требуется: вычислить M^k — результат возведения матрицы в степень k .

Ограничения:

- Числа в матрице не должны превышать 1000;
- Числа в матрице не должны быть меньше -1000.

5 Алгоритм решения задачи

Для решения задачи применялся алгоритм бинарного возведения в степень:

1. Если n четно, то верно $a^n = a^{n/2} \cdot a^{n/2} = (a^{n/2})^2$.
2. Если n нечетно, то верно $a^n = a^{n-1} \cdot a = (a^{\frac{n-1}{2}})^2 \cdot a$, то есть можно свести задачу к размеру $n - 1$ (к четной форме).

Возведение матрицы в степень — это ее многократное умножение саму на себя

$$A^n = \underbrace{A \cdot A \cdot \dots \cdot A}_{n \text{ раз}}$$

При этом для $n = 0$: $A^0 = E$, где E — единичная матрица.

Пусть даны две квадратные матрицы A и B размера $n \times n$:

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix}, \quad B = \begin{pmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{nn} \end{pmatrix}$$

Произведение матриц $C = A \times B$ есть матрица C размера $n \times n$, элементы которой вычисляются по формуле:

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{in}b_{nj}, \text{ где } i, j = 1, 2, \dots, n.$$

5.1 Метод распараллеливания алгоритма

Рассмотрим два подхода к распараллеливанию этой задачи на CUDA: с использованием глобальной памяти и с использованием разделяемой (shared) памяти.

5.1.1 Глобальная память

1. Распределение потоков

- Размер матрицы равен $n \times n$, тогда общее число элементов матрицы равно N . Запускается одномерная 1D-сетка потоков.

Каждый поток обрабатывает несколько элементов результирующей матрицы с шагом s , где

$$s = \frac{N}{gridDim.x \cdot blockDim.x},$$

где $gridDim.x$ - число блоков, а $blockDim.x$ - число потоков внутри одного блока.

- Индекс потока вычисляется как

$$threadId = blockIdx.x \cdot blockDim.x + threadIdx.x$$

- Каждый поток обрабатывает элементы с индексами

$$idx = threadId + i \cdot T, \quad i = 0, 1, 2, \dots, \text{ при условии } idx < N.$$

- Каждый поток копирует элементы между матрицами, используя только глобальную память.

2. Умножение матриц

- Координаты элементов вычисляются как:

$$row = \frac{idx}{n}$$

$$col = idx \bmod n$$

- Каждый поток вычисляет один элемент результирующей матрицы:

$$C[row][col] = \sum_{k=0}^{n-1} A[row][k] \cdot B[k][col]$$

5.1.2 Разделяемая память

1. Разбиение матрицы на блоки данных

- Запускается одномерная 1D-сетка потоков, которая делится на одномерные блоки потоков.
- В каждом блоке выделяется разделяемая память. Она разбивается на два буфера:

– $rowCacheA$ — для хранения части строки матрицы A ;

– $colCacheB$ — для хранения части столбца матрицы B .

- Размер каждого буфера составляет $blockDim.x$

2. Копирование данных в разделяемую память

- Каждый поток загружает свою часть элементов параллельно другим потокам.
- После загрузки данных производится их синхронизация. Это гарантирует, что все элементы матрицы будут помещены в разделяемую память до начала вычислений.
- После синхронизации каждый поток вычисляет частичную сумму скалярного произведения, используя значения из разделяемой памяти. После завершения вычислений выполняется повторная синхронизация.

3. Копирование данных в глобальную память

- После всех вычислений потоки параллельно копируют данные из разделяемой памяти в глобальную. Этот процесс аналогичен предыдущему этапу.

6 Описание эксперимента

Цель эксперимента - оценка зависимости времени решения задачи от объема обрабатываемых данных и параметров запуска, связанных с GPU.

Для исследования времени решения задачи в зависимости от параметров были выбраны следующие параметры:

- Используемая память: глобальная, разделяемая и глобальная.
- Количество блоков: 1, 10, 100, 1000, 10000.
- Количество потоков в блоке: 1, 10, 100, 1000.
- Размеры матрицы: 10 x 10, 100 x 100, 1000 x 1000.
- Количество элементов в матрице: 100, 10000, 1000000.

Для каждой комбинации параметров было проведено 100 измерений с помощью событий CUDA и взято среднее значение.

7 Анализ результатов

В таблицах 1-3 представлены усредненные результаты измерения возведения матрицы в степень 64 с использованием глобальной памяти.

Таблица 1. Усредненные результаты измерения времени исполнения программы для матрицы 10×10 и глобальной памяти (мс)

Число потоков в блоке	Число блоков				
	1	10	100	1000	10000
1	0.084	0.067	0.068	0.066	0.063
10	0.057	0.068	0.056	0.062	0.057
100	0.065	0.065	0.063	0.062	0.065
1000	0.065	0.064	0.066	0.061	0.066

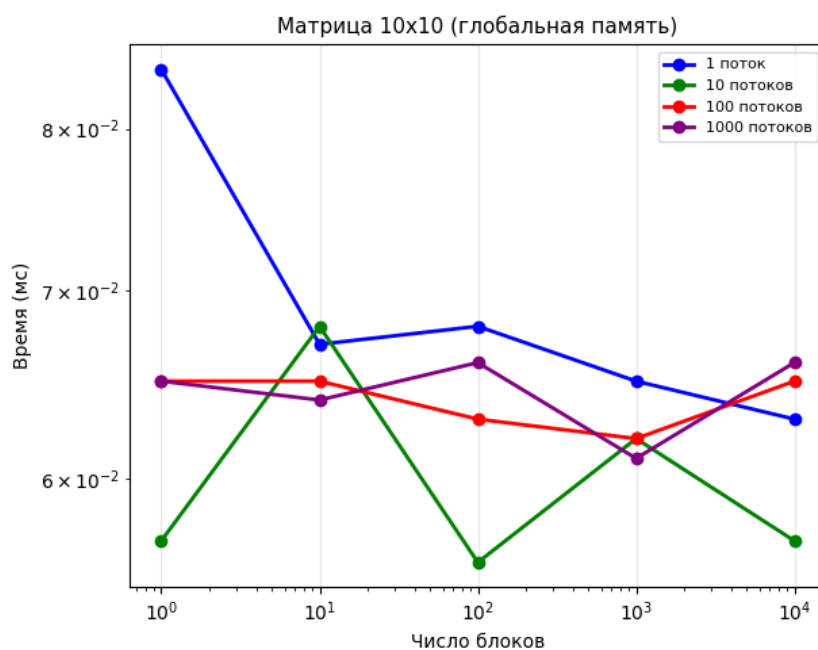


Рис. 6. График результатов времени исполнения программы для матрицы 10×10

Таблица 2. Усредненные результаты измерения времени исполнения программы для матрицы 100×100 и глобальной памяти (мс)

Число потоков в блоке	Число блоков				
	1	10	100	1000	10000
1	5.158	5.164	5.155	5.155	5.168
10	0.698	0.684	0.683	0.686	0.683
100	0.300	0.297	0.295	0.293	0.291
1000	0.343	0.339	0.341	0.341	0.334

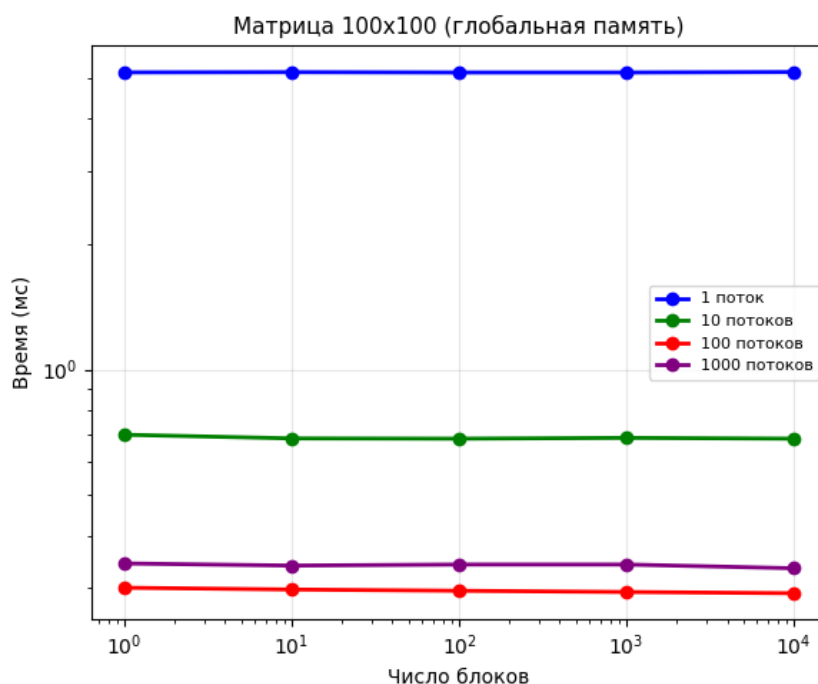


Рис. 7. График результатов времени исполнения программы для матрицы 100×100

Таблица 3. Усредненные результаты измерения времени исполнения программы для матрицы 1000×1000 и глобальной памяти (мс)

Число потоков в блоке	Число блоков				
	1	10	100	1000	10000
1	6457.359	6456.862	6455.586	6455.680	6455.478
10	700.153	700.082	700.053	700.035	700.082
100	267.128	267.536	267.204	267.459	267.320
1000	203.149	204.927	203.521	204.870	205.113

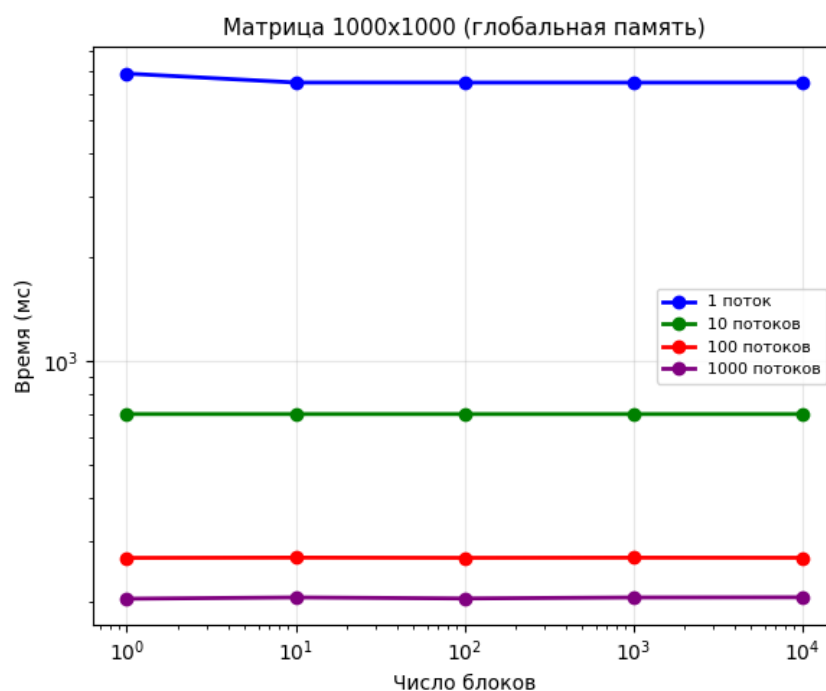


Рис. 8. График результатов времени исполнения программы для матрицы 1000×1000

В таблицах 4-6 представлены усредненные результаты измерения возведения матрицы в степень 64 с использованием глобальной и разделяемой памяти.

Таблица 4. Усредненные результаты измерения времени исполнения программы для матрицы 10×10 и глобальной+разделяемой памяти (мс)

Число потоков в блоке	Число блоков				
	1	10	100	1000	10000
1	0.195	0.094	0.092	0.091	0.286
10	0.109	0.095	0.095	0.098	0.256
100	0.097	0.098	0.155	0.099	0.286
1000	0.093	0.096	0.101	0.180	1.104

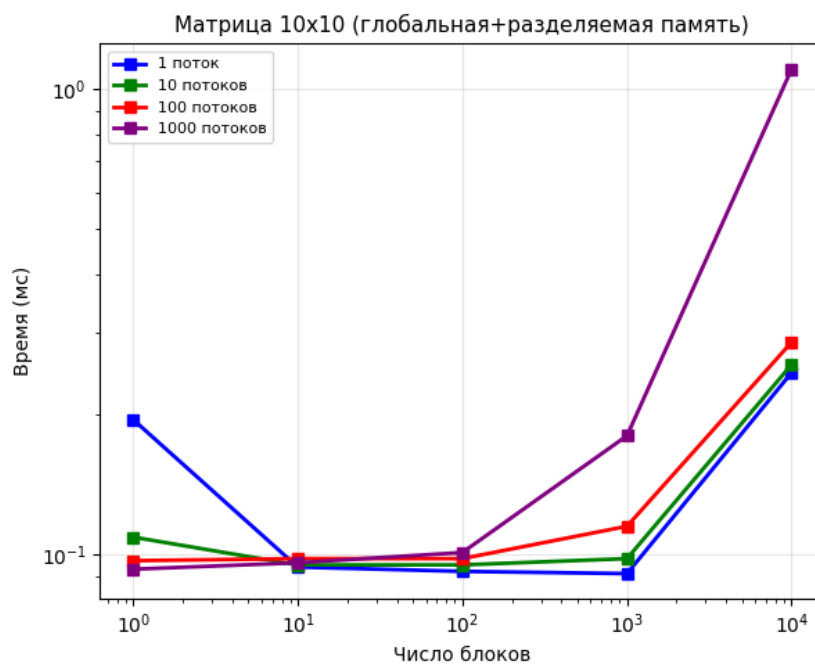


Рис. 9. График результатов времени исполнения программы для матрицы 10×10

Таблица 5. Усредненные результаты измерения времени исполнения программы для матрицы 100×100 и глобальной+разделяемой памяти (мс)

Число потоков в блоке	Число блоков				
	1	10	100	1000	10000
1	0.188	0.184	0.231	0.731	4.766
10	0.185	0.192	0.192	0.661	0.735
100	0.210	0.214	0.346	0.296	0.300
1000	0.363	0.370	0.369	0.374	1.220

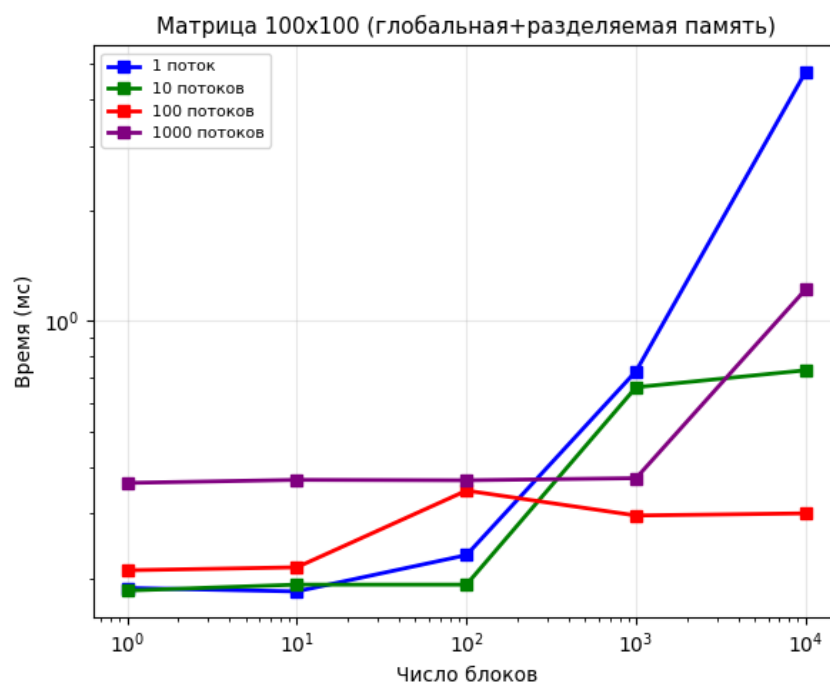


Рис. 10. График результатов времени исполнения программы для матрицы 100×100

Таблица 6. Усредненные результаты измерения времени исполнения программы для матрицы 1000×1000 и глобальной+разделяемой памяти (мс)

Число потоков в блоке	Число блоков				
	1	10	100	1000	10000
1	1.338	1.292	1.323	6.752	56.191
10	1.363	1.347	1.582	7.266	70.151
100	1.597	1.614	2.703	26.304	273.540
1000	2.972	2.918	20.302	205.127	204.668

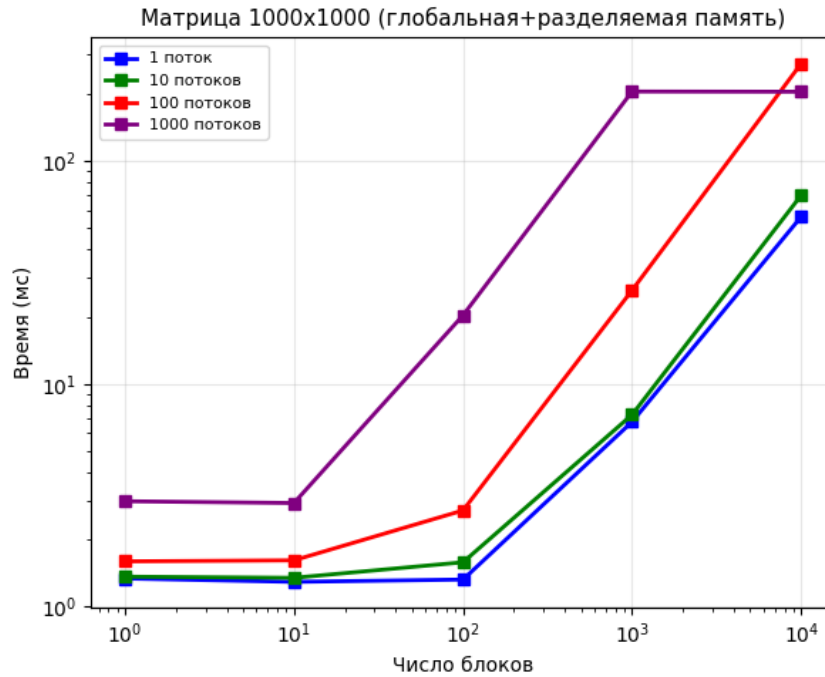


Рис. 11. График результатов времени исполнения программы для матрицы 1000×1000

Зависимость от размера матрицы

Для всех матриц время выполнения увеличивается с увеличением размера матрицы, так как становится больше данных и больше вычислений. Для малых размеров матрицы значительную часть времени составляют накладные расходы.

- При малых размерах $n = 100$ минимальное время составляет 0.056 мс (Таблица 1).
- При средних размерах $n = 1000$ минимальное время составляет 0.184 мс (Таблица 5).
- При больших размерах $n = 10000$ минимальное время составляет 1.292 мс (Таблица 6).

Зависимость от числа блоков и потоков

Количество потоков в блоке больше влияет на производительность, чем количество блоков. Рассмотрим на примере большой матрицы 1000×1000 (таблица 3):

- При 1 потоке в блоке каждый поток вынужден обрабатывать N

элементов последовательно, что приводит к минимальному использованию параллелизма и большому времени выполнения, что особенно заметно на больших матрицах. Для матрицы 1000×1000 это даёт время около 6457 мс.

- При увеличении до 10 потоков время снижается в 9-10 раз (около 700 мс).
- При 100 и 1000 потоках дальнейшее увеличение даёт меньший прирост (около 203-267 мс), так как пропускная способность глобальной памяти ограничена.

Количество блоков меньше влияет на производительность. Для всех размеров матриц изменение числа блоков при фиксированном числе потоков даёт разброс времени не более 5-10%. Например, для матрицы 100×100 при числе потоков=10 время выполнения составляет от 0.683 мс до 0.698 мс.

Наименьшее время достигается при оптимальном соотношении количества потоков и блоков в них. Например, для матрицы 100 элементов (10 на 10) минимальное время (0.056 мс) достигается при 100 блоках и 10 потоках в каждом блоке.

Сравнение глобальной и разделяемой памяти

Результаты сравнения двух подходов к организации памяти представлены в таблице 7.

- Для малых матриц (10×10) разделяемая память не даёт преимущества, а в некоторых конфигурациях показывает худшие результаты. Это связано с тем, что время, которое тратится на накладные расходы (загрузка данных в разделяемую память, синхронизация потоков), превышает выгоду от ускоренного доступа. При малом объёме данных задержки глобальной памяти практически не проявляются.
- Для средних матриц (100×100) разделяемая память даёт небольшое улучшение в некоторых конфигурациях. Выгода от повторного использования элементов в разделяемой памяти начинает компенсировать накладные расходы, которые на нее необходимы.
- Для больших матриц (1000×1000) разделяемая память показывает значительно лучшие результаты, чем глобальная. Это связано с

временем доступа: к разделяемой памяти он намного быстрее, чем к глобальной, что при многократном использовании данных дает значимый эффект.

Результаты сравнения двух подходов к организации памяти представлены в таблице 7.

Таблица 7. Таблица максимального и минимального времени для каждой конфигурации

размер матрицы	тип памяти	мин/макс время	время (мс)	блоки	потoki
10x10	глобальная	максимальное	0.084	1	1
		минимальное	0.056	100	10
	глобальная + разделяемая	максимальное	1.104	10000	1000
		минимальное	0.091	1000	1
100x100	глобальная	максимальное	5.164	10	1
		минимальное	0.291	10000	100
	глобальная + разделяемая	максимальное	4.766	10000	1
		минимальное	0.184	10	1
1000x1000	глобальная	максимальное	6457.359	1	1
		минимальное	203.149	1	1000
	глобальная + разделяемая	максимальное	273.668	10000	100
		минимальное	1.292	10	1

Заключение

В ходе данной курсовой работы мной было изучена технология параллельного программирования на основе архитектуры Nvidia CUDA.

В теоретической части работы был проведен обзор архитектуры NVIDIA CUDA, включающий обзор её вычислительных возможностей, модели памяти и модели вычислений на графических процессорах NVIDIA.

В ходе практической части работы:

- Реализована и распараллелена программа на языке CUDA C для возведения матрицы в k -тую степень.
- Первично программа была отлажена на персональном компьютере, поддерживающем технологию NVIDIA.
- Финальная серия экспериментов была проведена на суперкомпьютере «Политехнический» с использованием вычислительного узла «Политехник - РСК Торнадо» (графический ускоритель NVIDIA K40).
- В ходе серии экспериментов были исследованы различные конфигурации параметров распараллеливания: изменялся размер исходной матрицы, количество блоков в сетке, количество потоков в блоке и используемые типы памяти.
- В результате экспериментов были получены 120 усредненных измерений.
- Проведен анализ полученных результатов.

В ходе анализа эксперимента было выявлено, что разделяемая память дает значительное ускорение при больших объемах данных, требующих быстрого доступа к памяти. При малых объемах данных эффективнее работает глобальная память.

Измерение времени работы показало, что производительность зависит от числа потоков, количества потоков в блоке и типа используемой памяти. При использовании глобальной памяти минимальное время выполнения достигается, когда произведение числа блоков на число потоков в блоке приблизительно равно количеству элементов матрицы (например, в матрице 1000×1000 оптимальная конфигурация - 1 блок на 1000 потоков). При использовании глобальной и разделяемой памяти оптимальная производительность достигается при умеренном числе блоков

(10-100) и достаточном числе потоков для эффективного использования разделяемой памяти. Для матрицы 1000×1000 минимальное время 1.292 мс получено при 10 блоках и 1 потоке, что объясняется многократным использованием данных, загруженных в разделяемую память, и снижением обращений к глобальной памяти.

Список литературы

- [1] Сандерс, Д. Технология CUDA в примерах: введение в программирование графических процессоров. — М: ДМК Пресс, 2013 г — 232 с.
- [2] Краткое руководство пользователя вычислителей «Политехник - РСК Торнадо» и «Политехник - РСК Петастрим».
- [3] Суперкомпьютерный центр - Ресурсы СКЦ. // Суперкомпьютерный центр «Политехнический». [Электронный ресурс]. – Режим доступа: URL: <https://scc.spbstu.ru/resources> (дата обращения: 20.03.2026).